

Abstract Syntax Tree

Generated by Doxygen 1.8.11

Contents

1	Data Structure Index	1
1.1	Data Structures	1
2	File Index	3
2.1	File List	3
3	Data Structure Documentation	5
3.1	_actionNode Struct Reference	5
3.1.1	Detailed Description	5
3.1.2	Field Documentation	6
3.1.2.1	expr	6
3.1.2.2	name	6
3.1.2.3	next	6
3.2	_etapeNode Struct Reference	6
3.2.1	Detailed Description	8
3.2.2	Field Documentation	8
3.2.2.1	actions	8
3.2.2.2	active	8
3.2.2.3	id	8
3.2.2.4	name	8
3.2.2.5	nbTransitionsAmont	8
3.2.2.6	nbTransitionsAval	8
3.2.2.7	next	8
3.2.2.8	transitionsAmont	8

3.2.2.9	transitionsAval	9
3.3	_etapeRefNode Struct Reference	9
3.3.1	Detailed Description	9
3.3.2	Field Documentation	10
3.3.2.1	etape	10
3.3.2.2	name	10
3.3.2.3	next	10
3.4	_transitionNode Struct Reference	10
3.4.1	Detailed Description	11
3.4.2	Field Documentation	11
3.4.2.1	etapesAmont	11
3.4.2.2	etapesAval	11
3.4.2.3	id	11
3.4.2.4	name	11
3.4.2.5	nbEtapesAmont	11
3.4.2.6	nbEtapesAval	11
3.4.2.7	next	11
3.4.2.8	receptivite	12
3.5	_transitionRefNode Struct Reference	12
3.5.1	Detailed Description	12
3.5.2	Field Documentation	13
3.5.2.1	name	13
3.5.2.2	next	13
3.5.2.3	transition	13
3.6	identifierNodeType Struct Reference	13
3.6.1	Detailed Description	13
3.6.2	Field Documentation	13
3.6.2.1	flag	13
3.6.2.2	ident	13
3.6.2.3	key	14

3.7	nodeTypeTag Struct Reference	14
3.7.1	Detailed Description	14
3.7.2	Field Documentation	15
3.7.2.1	"@1	15
3.7.2.2	digraphNode	15
3.7.2.3	t_identifier	15
3.7.2.4	t_numeric	15
3.7.2.5	t_oper	15
3.7.2.6	type	15
3.8	numericNodeType Struct Reference	15
3.8.1	Detailed Description	15
3.8.2	Field Documentation	16
3.8.2.1	valeur	16
3.9	operatorNodeType Struct Reference	16
3.9.1	Detailed Description	16
3.9.2	Field Documentation	16
3.9.2.1	nOperands	16
3.9.2.2	op	17
3.9.2.3	oper	17
4	File Documentation	19
4.1	ast.c File Reference	19
4.1.1	Detailed Description	20
4.1.2	Function Documentation	20
4.1.2.1	createIdentifierNode(char *id, int flag)	20
4.1.2.2	createNumericNode(float v)	20
4.1.2.3	createOperatorNode(int oper, int nops,...)	20
4.1.2.4	Extract_Num(char *str)	21
4.1.2.5	findEtapeByName(etapeNode *head, char *n)	21
4.1.2.6	findTransitionByName(transitionNode *head, char *n)	21
4.1.2.7	generateAsmExpression(nodeType *n, FILE *fout)	21

4.1.2.8	<code>generateCode(etapeNode *en, transitionNode *tn)</code>	21
4.1.2.9	<code>makeDual(etapeNode *en, transitionNode *tn)</code>	21
4.1.2.10	<code>nbEtapes(etapeNode *head)</code>	21
4.1.2.11	<code>nbTransitions(transitionNode *head)</code>	21
4.1.2.12	<code>printEtapes(etapeNode *en)</code>	21
4.1.2.13	<code>printRefEtapes(etapeRefNode *head)</code>	21
4.1.2.14	<code>printRefTransitions(transitionRefNode *head)</code>	22
4.1.2.15	<code>printTransitions(transitionNode *tn)</code>	22
4.1.2.16	<code>pushAction(actionNode **head, actionNode *n)</code>	22
4.1.2.17	<code>pushEtape(etapeNode **head, etapeNode *n)</code>	22
4.1.2.18	<code>pushRefEtape(etapeRefNode **head, char *n)</code>	22
4.1.2.19	<code>pushRefTransition(transitionRefNode **head, char *n)</code>	22
4.1.2.20	<code>pushTransition(transitionNode **head, transitionNode *n)</code>	22
4.1.3	Variable Documentation	22
4.1.3.1	<code>compteurEtapes</code>	22
4.1.3.2	<code>compteurLabel</code>	22
4.1.3.3	<code>num_variable</code>	23
4.2	ast.h File Reference	23
4.2.1	Detailed Description	25
4.2.2	Macro Definition Documentation	25
4.2.2.1	<code>KEY_ANA</code>	25
4.2.2.2	<code>KEY_BUTTON</code>	25
4.2.2.3	<code>KEY_IDENT</code>	25
4.2.2.4	<code>KEY_PWM</code>	25
4.2.2.5	<code>KEY_TOR</code>	25
4.2.3	Typedef Documentation	25
4.2.3.1	<code>actionNode</code>	25
4.2.3.2	<code>etapeNode</code>	25
4.2.3.3	<code>etapeRefNode</code>	25
4.2.3.4	<code>nodeType</code>	25

4.2.3.5	transitionNode	26
4.2.3.6	transitionRefNode	26
4.2.4	Enumeration Type Documentation	26
4.2.4.1	nodeEnum	26
4.2.4.2	nodeOper	26
4.2.5	Function Documentation	27
4.2.5.1	createIdentifierNode(char *id, int flag)	27
4.2.5.2	createNumericNode(float v)	27
4.2.5.3	createOperatorNode(int oper, int nOperands,...)	27
4.2.5.4	Extract_Num(char *str)	27
4.2.5.5	findEtapeByName(etapeNode *head, char *s)	27
4.2.5.6	generateAsmExpression(nodeType *n, FILE *fout)	27
4.2.5.7	generateCode(etapeNode *en, transitionNode *tn)	27
4.2.5.8	grafcet_interpreter()	27
4.2.5.9	makeDual(etapeNode *en, transitionNode *tn)	27
4.2.5.10	printEtapes(etapeNode *head)	27
4.2.5.11	printRefEtapes(etapeRefNode *head)	28
4.2.5.12	printRefTransitions(transitionRefNode *head)	28
4.2.5.13	printTransitions(transitionNode *head)	28
4.2.5.14	pushAction(actionNode **head, actionNode *action)	28
4.2.5.15	pushEtape(etapeNode **head, etapeNode *etape)	28
4.2.5.16	pushRefEtape(etapeRefNode **head, char *n)	28
4.2.5.17	pushRefTransition(transitionRefNode **head, char *n)	28
4.2.5.18	pushTransition(transitionNode **head, transitionNode *trans)	28
Index		29

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

_actionNode	5
_etapeNode	6
_etapeRefNode	9
_transitionNode	10
_transitionRefNode	12
identifierNodeType	13
nodeTypeTag	14
numericNodeType	15
operatorNodeType	16

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

ast.c	Ce code permet de générer un fichier asm puis un fichier bin à partir d'un fichier grafcet	19
ast.h	Librairie associée au fichier ast.c	23

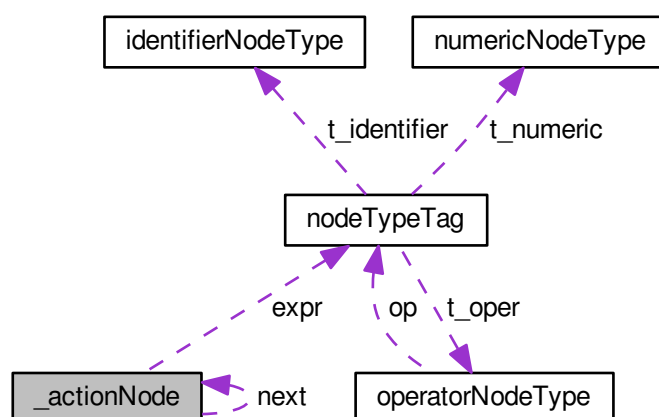
Chapter 3

Data Structure Documentation

3.1 `_actionNode` Struct Reference

```
#include <ast.h>
```

Collaboration diagram for `_actionNode`:



Data Fields

- `char * name`
- `nodeType * expr`
- `struct _actionNode * next`

3.1.1 Detailed Description

Definition at line 73 of file `ast.h`.

3.1.2 Field Documentation

3.1.2.1 `nodeType*` `expr`

Definition at line 76 of file `ast.h`.

3.1.2.2 `char*` `name`

Definition at line 75 of file `ast.h`.

3.1.2.3 `struct _actionNode*` `next`

Definition at line 78 of file `ast.h`.

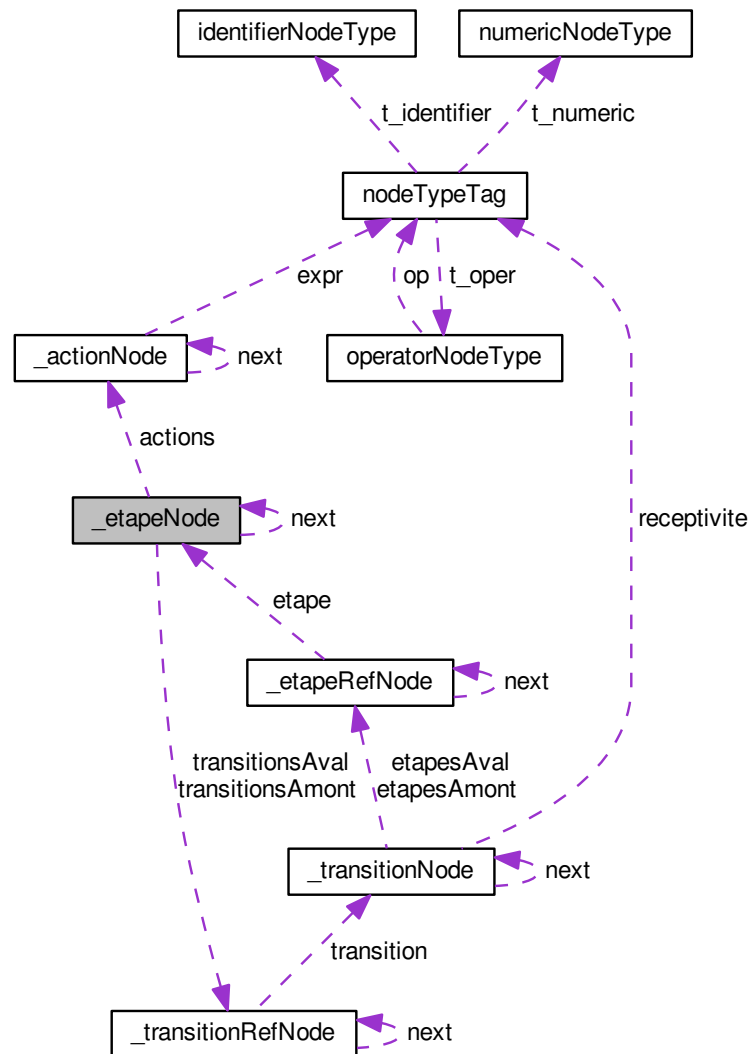
The documentation for this struct was generated from the following file:

- [ast.h](#)

3.2 `_etapeNode` Struct Reference

```
#include <ast.h>
```

Collaboration diagram for _etapeNode:



Data Fields

- char * [name](#)
- int [id](#)
- int [active](#)
- struct `_actionNode` * [actions](#)
- struct `_transitionRefNode` * [transitionsAmont](#)
- int [nbTransitionsAmont](#)
- struct `_transitionRefNode` * [transitionsAval](#)
- int [nbTransitionsAval](#)
- struct `_etapeNode` * [next](#)

3.2.1 Detailed Description

Definition at line 95 of file ast.h.

3.2.2 Field Documentation

3.2.2.1 struct **_actionNode*** actions

Definition at line 100 of file ast.h.

3.2.2.2 int active

Definition at line 99 of file ast.h.

3.2.2.3 int id

Definition at line 98 of file ast.h.

3.2.2.4 char* name

Definition at line 97 of file ast.h.

3.2.2.5 int nbTransitionsAmont

Definition at line 102 of file ast.h.

3.2.2.6 int nbTransitionsAval

Definition at line 104 of file ast.h.

3.2.2.7 struct **_etapeNode*** next

Definition at line 106 of file ast.h.

3.2.2.8 struct **_transitionRefNode*** transitionsAmont

Definition at line 101 of file ast.h.

3.2.2.9 struct _transitionRefNode* transitionsAval

Definition at line 103 of file ast.h.

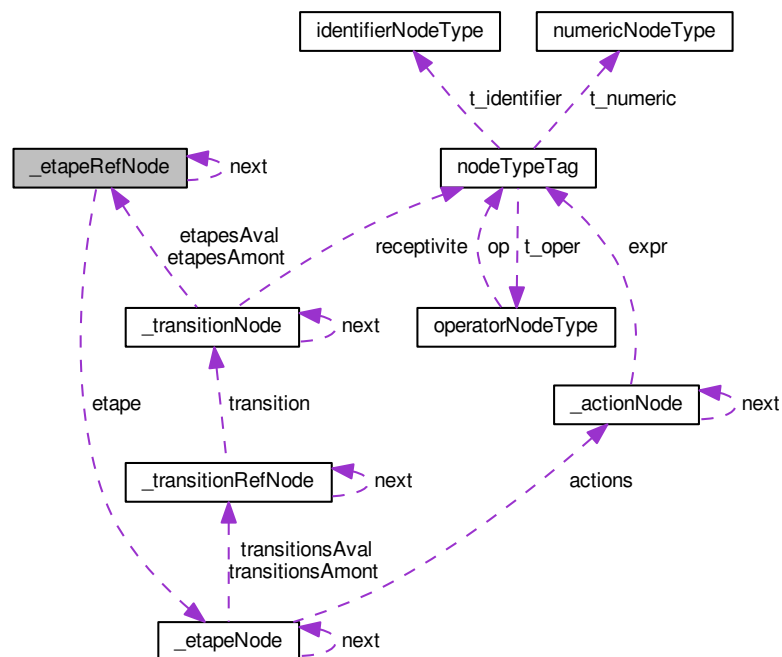
The documentation for this struct was generated from the following file:

- [ast.h](#)

3.3 _etapeRefNode Struct Reference

```
#include <ast.h>
```

Collaboration diagram for _etapeRefNode:



Data Fields

- char * [name](#)
- struct [_etapeNode](#) * [etape](#)
- struct [_etapeRefNode](#) * [next](#)

3.3.1 Detailed Description

Definition at line 81 of file ast.h.

3.3.2 Field Documentation

3.3.2.1 struct _etapeNode* etape

Definition at line 84 of file ast.h.

3.3.2.2 char* name

Definition at line 83 of file ast.h.

3.3.2.3 struct _etapeRefNode* next

Definition at line 85 of file ast.h.

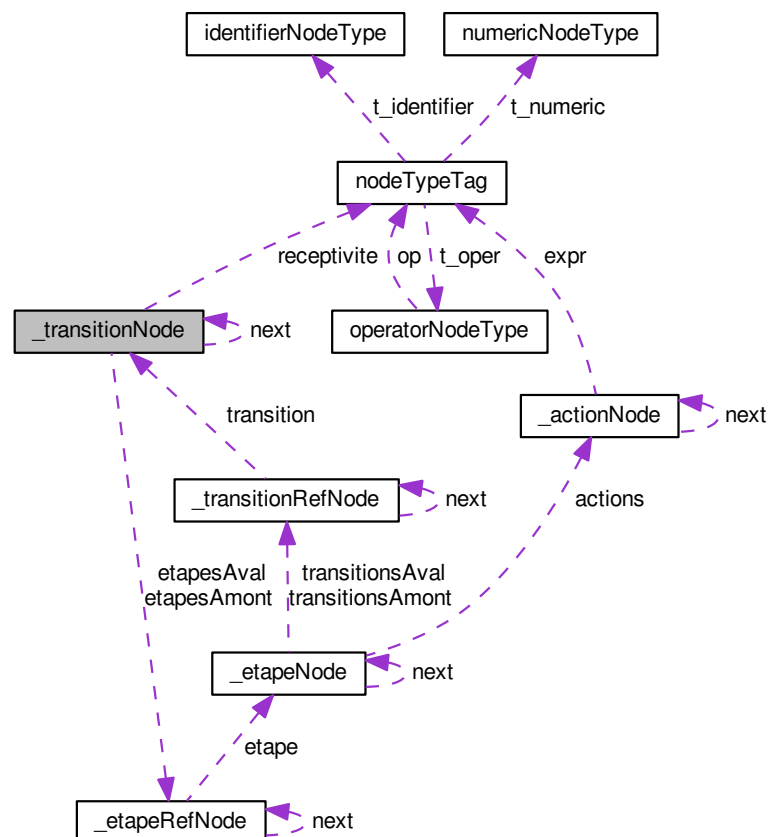
The documentation for this struct was generated from the following file:

- [ast.h](#)

3.4 _transitionNode Struct Reference

```
#include <ast.h>
```

Collaboration diagram for _transitionNode:



Data Fields

- `char * name`
- `int id`
- `struct _etapeRefNode * etapesAmont`
- `int nbEtapesAmont`
- `struct _etapeRefNode * etapesAval`
- `int nbEtapesAval`
- `nodeType * receptivite`
- `struct _transitionNode * next`

3.4.1 Detailed Description

Definition at line 109 of file `ast.h`.

3.4.2 Field Documentation

3.4.2.1 `struct _etapeRefNode* etapesAmont`

Definition at line 113 of file `ast.h`.

3.4.2.2 `struct _etapeRefNode* etapesAval`

Definition at line 115 of file `ast.h`.

3.4.2.3 `int id`

Definition at line 112 of file `ast.h`.

3.4.2.4 `char* name`

Definition at line 111 of file `ast.h`.

3.4.2.5 `int nbEtapesAmont`

Definition at line 114 of file `ast.h`.

3.4.2.6 `int nbEtapesAval`

Definition at line 116 of file `ast.h`.

3.4.2.7 `struct _transitionNode* next`

Definition at line 119 of file `ast.h`.

3.5.2 Field Documentation

3.5.2.1 char* name

Definition at line 90 of file ast.h.

3.5.2.2 struct _transitionRefNode* next

Definition at line 92 of file ast.h.

3.5.2.3 struct _transitionNode* transition

Definition at line 91 of file ast.h.

The documentation for this struct was generated from the following file:

- [ast.h](#)

3.6 identifierNodeType Struct Reference

```
#include <ast.h>
```

Data Fields

- char * [ident](#)
- int [key](#)
- int [flag](#)

3.6.1 Detailed Description

un noeud identifier fait reference a un identificateur

Definition at line 47 of file ast.h.

3.6.2 Field Documentation

3.6.2.1 int flag

Definition at line 50 of file ast.h.

3.6.2.2 char* ident

Definition at line 48 of file ast.h.

3.6.2.3 int key

Definition at line 49 of file ast.h.

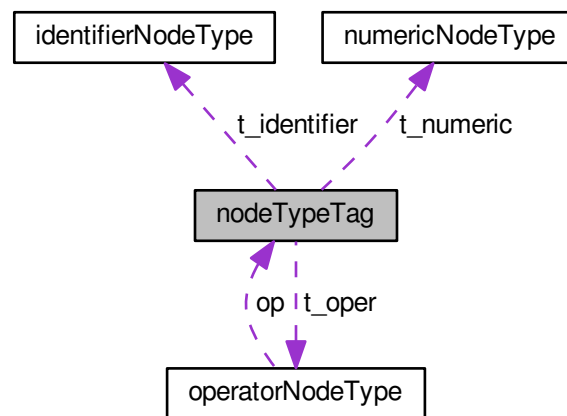
The documentation for this struct was generated from the following file:

- [ast.h](#)

3.7 nodeTypeTag Struct Reference

```
#include <ast.h>
```

Collaboration diagram for nodeTypeTag:



Data Fields

- `nodeEnum` type
- int `digraphNode`
- union {
 - `numericNodeType t_numeric`
 - `operatorNodeType t_oper`
 - `identifierNodeType t_identifier`

3.7.1 Detailed Description

Cette structure, fondamentale, permet de définir un noeud a geometrie variable. Le type permet de définir le type du noeud (numeric ou operator) et l'union permet, en fonction du type, d'accéder aux bonnes informations

Definition at line 57 of file ast.h.

3.7.2 Field Documentation

3.7.2.1 union { ... }

3.7.2.2 int digraphNode

Definition at line 59 of file ast.h.

3.7.2.3 identifierNodeType t_identifier

Definition at line 63 of file ast.h.

3.7.2.4 numericNodeType t_numeric

Definition at line 61 of file ast.h.

3.7.2.5 operatorNodeType t_oper

Definition at line 62 of file ast.h.

3.7.2.6 nodeEnum type

Definition at line 58 of file ast.h.

The documentation for this struct was generated from the following file:

- [ast.h](#)

3.8 numericNodeType Struct Reference

```
#include <ast.h>
```

Data Fields

- int [valeur](#)

3.8.1 Detailed Description

un noeud numeric (float) permet de stocker une valeur reelle

Definition at line 34 of file ast.h.

3.8.2 Field Documentation

3.8.2.1 int valeur

Definition at line 35 of file ast.h.

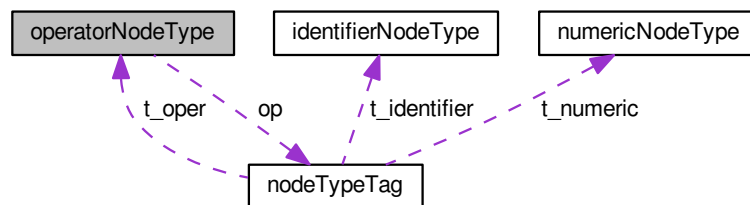
The documentation for this struct was generated from the following file:

- [ast.h](#)

3.9 operatorNodeType Struct Reference

```
#include <ast.h>
```

Collaboration diagram for operatorNodeType:



Data Fields

- int [oper](#)
- int [nOperands](#)
- struct [nodeTypeTag](#) ** [op](#)

3.9.1 Detailed Description

un noeud operator permet de stocker un operateur, le nombre d'operandes et les pointeurs vers les noeuds operande correspondants

Definition at line 40 of file ast.h.

3.9.2 Field Documentation

3.9.2.1 int nOperands

Definition at line 42 of file ast.h.

3.9.2.2 struct nodeTypeTag** op

Definition at line 43 of file ast.h.

3.9.2.3 int oper

Definition at line 41 of file ast.h.

The documentation for this struct was generated from the following file:

- [ast.h](#)

Chapter 4

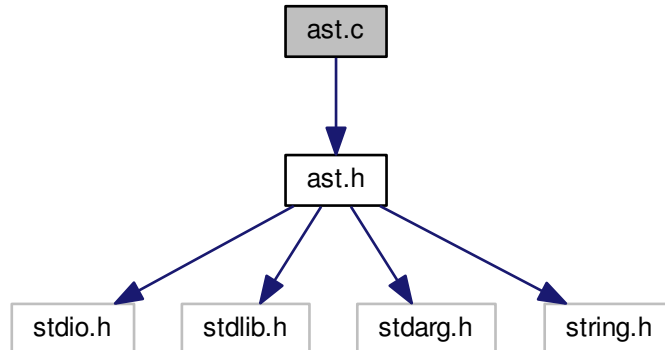
File Documentation

4.1 ast.c File Reference

Ce code permet de générer un fichier asm puis un fichier bin à partir d'un fichier grafcet.

```
#include "ast.h"
```

Include dependency graph for ast.c:



Functions

- `nodeType * createNumericNode (float v)`
- `nodeType * createOperatorNode (int oper, int nops,...)`
- `nodeType * createIdentifierNode (char *id, int flag)`
- `void generateAsmExpression (nodeType *n, FILE *fout)`
- `void pushRefEtape (etapeRefNode **head, char *n)`
- `void pushRefTransition (transitionRefNode **head, char *n)`
- `void pushAction (actionNode **head, actionNode *n)`
- `void pushEtape (etapeNode **head, etapeNode *n)`
- `void pushTransition (transitionNode **head, transitionNode *n)`

- void `printRefEtapes` (`etapeRefNode` *head)
- void `printRefTransitions` (`transitionRefNode` *head)
- int `nbEtapes` (`etapeNode` *head)
- int `nbTransitions` (`transitionNode` *head)
- `etapeNode` * `findEtapeByName` (`etapeNode` *head, char *n)
- `transitionNode` * `findTransitionByName` (`transitionNode` *head, char *n)
- void `printEtapes` (`etapeNode` *en)
- void `printTransitions` (`transitionNode` *tn)
- void `makeDual` (`etapeNode` *en, `transitionNode` *tn)
- int `Extract_Num` (char *str)
- void `generateCode` (`etapeNode` *en, `transitionNode` *tn)

Variables

- int `compteurLabel` =0
- int `compteurEtapes` =0
- int `num_variable`

4.1.1 Detailed Description

Ce code permet de générer un fichier asm puis un fichier bin à partir d'un fichier grafcet.

[ast.c](#)

Author

Jeanne Baumier & Thomas Broussard

Date

12 juin 2017

4.1.2 Function Documentation

4.1.2.1 `nodeType*` `createIdentifierNode` (char * *id*, int *flag*)

Definition at line 86 of file ast.c.

4.1.2.2 `nodeType*` `createNumericNode` (float *v*)

Definition at line 30 of file ast.c.

4.1.2.3 `nodeType*` `createOperatorNode` (int *oper*, int *nops*, ...)

Definition at line 57 of file ast.c.

4.1.2.4 int Extract_Num (char * *str*)

Definition at line 598 of file ast.c.

4.1.2.5 etapeNode* findEtapeByName (etapeNode * *head*, char * *n*)

Definition at line 374 of file ast.c.

4.1.2.6 transitionNode* findTransitionByName (transitionNode * *head*, char * *n*)

Definition at line 388 of file ast.c.

4.1.2.7 void generateAsmExpression (nodeType * *n*, FILE * *fout*)

Definition at line 113 of file ast.c.

4.1.2.8 void generateCode (etapeNode * *en*, transitionNode * *tn*)

Definition at line 629 of file ast.c.

4.1.2.9 void makeDual (etapeNode * *en*, transitionNode * *tn*)

Definition at line 448 of file ast.c.

4.1.2.10 int nbEtapes (etapeNode * *head*)

Definition at line 348 of file ast.c.

4.1.2.11 int nbTransitions (transitionNode * *head*)

Definition at line 361 of file ast.c.

4.1.2.12 void printEtapes (etapeNode * *en*)

Definition at line 402 of file ast.c.

4.1.2.13 void printRefEtapes (etapeRefNode * *head*)

Definition at line 316 of file ast.c.

4.1.2.14 void printRefTransitions (transitionRefNode * head)

Definition at line 328 of file ast.c.

4.1.2.15 void printTransitions (transitionNode * tn)

Definition at line 428 of file ast.c.

4.1.2.16 void pushAction (actionNode ** head, actionNode * n)

Definition at line 265 of file ast.c.

4.1.2.17 void pushEtape (etapeNode ** head, etapeNode * n)

Definition at line 282 of file ast.c.

4.1.2.18 void pushRefEtape (etapeRefNode ** head, char * n)

Definition at line 219 of file ast.c.

4.1.2.19 void pushRefTransition (transitionRefNode ** head, char * n)

Definition at line 242 of file ast.c.

4.1.2.20 void pushTransition (transitionNode ** head, transitionNode * n)

Definition at line 299 of file ast.c.

4.1.3 Variable Documentation**4.1.3.1 int compteurEtapas =0**

compteurEtapas permet de compter le nombre de Labels

Definition at line 17 of file ast.c.

4.1.3.2 int compteurLabel =0

compteurLabel permet de compter le nombre de Labels

Definition at line 15 of file ast.c.

4.1.3.3 int num_variable

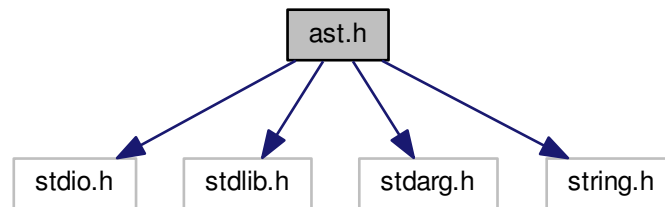
Definition at line 19 of file ast.c.

4.2 ast.h File Reference

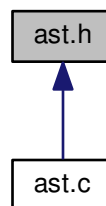
Librairie associée au fichier [ast.c](#).

```
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
```

Include dependency graph for ast.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [numericNodeType](#)
- struct [operatorNodeType](#)
- struct [identifierNodeType](#)
- struct [nodeTypeTag](#)
- struct [_actionNode](#)
- struct [_etapeRefNode](#)
- struct [_transitionRefNode](#)
- struct [_etapeNode](#)
- struct [_transitionNode](#)

Macros

- `#define KEY_BUTTON 0`
- `#define KEY_ANA 1`
- `#define KEY_PWM 2`
- `#define KEY_TOR 3`
- `#define KEY_IDENT 4`

Typedefs

- `typedef struct nodeTypeTag nodeType`
- `typedef struct _actionNode actionNode`
- `typedef struct _etapeRefNode etapeRefNode`
- `typedef struct _transitionRefNode transitionRefNode`
- `typedef struct _etapeNode etapeNode`
- `typedef struct _transitionNode transitionNode`

Enumerations

- `enum nodeOper {`
`OPER_ADD, OPER_SUB, OPER_MULT, OPER_DIV,`
`OPER_SEQUENCE, OPER_TRANSITION, OPER_STEP, OPER_INF,`
`OPER_SUP, OPER_GE, OPER_LE, OPER_NE,`
`OPER_EQ, OPER_NOT, OPER_AND, OPER_OR,`
`OPER_NEG, OPER_ASSIGN, OPER_RESERVE_SPACE, OPER_MAIN }`
- `enum nodeEnum { typeNumeric, typeOperator, typeIdentifier }`

Functions

- `nodeType * createNumericNode (float v)`
- `nodeType * createIdentifierNode (char *id, int flag)`
- `nodeType * createOperatorNode (int oper, int nOperands,...)`
- `void generateAsmExpression (nodeType *n, FILE *fout)`
- `void grafcet_interpreter ()`
- `void pushRefEtape (etapeRefNode **head, char *n)`
- `void pushRefTransition (transitionRefNode **head, char *n)`
- `void pushAction (actionNode **head, actionNode *action)`
- `void pushEtape (etapeNode **head, etapeNode *etape)`
- `void pushTransition (transitionNode **head, transitionNode *trans)`
- `etapeNode * findEtapeByName (etapeNode *head, char *s)`
- `void printRefEtapes (etapeRefNode *head)`
- `void printRefTransitions (transitionRefNode *head)`
- `void printEtapes (etapeNode *head)`
- `void printTransitions (transitionNode *head)`
- `void makeDual (etapeNode *en, transitionNode *tn)`
- `void generateCode (etapeNode *en, transitionNode *tn)`
- `int Extract_Num (char *str)`

4.2.1 Detailed Description

Librairie associée au fichier [ast.c](#).

[ast.h](#)

Author

Jeanne Baumier & Thomas Broussard

Date

12 juin 2017

4.2.2 Macro Definition Documentation

4.2.2.1 `#define KEY_ANA 1`

Definition at line 18 of file `ast.h`.

4.2.2.2 `#define KEY_BUTTON 0`

Definition at line 17 of file `ast.h`.

4.2.2.3 `#define KEY_IDENT 4`

Definition at line 21 of file `ast.h`.

4.2.2.4 `#define KEY_PWM 2`

Definition at line 19 of file `ast.h`.

4.2.2.5 `#define KEY_TOR 3`

Definition at line 20 of file `ast.h`.

4.2.3 Typedef Documentation

4.2.3.1 `typedef struct _actionNode actionNode`

4.2.3.2 `typedef struct _etapeNode etapeNode`

4.2.3.3 `typedef struct _etapeRefNode etapeRefNode`

4.2.3.4 `typedef struct nodeTypeTag nodeType`

Cette structure, fondamentale, permet de definir un noeud a geometrie variable. Le type permet de definir le type du noeud (numeric ou operator) et l'union permet, en fonction du type, d'accéder aux bonnes informations

4.2.3.5 `typedef struct _transitionNode transitionNode`

4.2.3.6 `typedef struct _transitionRefNode transitionRefNode`

4.2.4 Enumeration Type Documentation

4.2.4.1 `enum nodeEnum`

Les deux types de noeuds possibles, "numeric" ou "operator"

Enumerator

typeNumeric
typeOperator
typeIdentifier

Definition at line 31 of file ast.h.

4.2.4.2 `enum nodeOper`

Cet enum contient tous les types possibles pour un noeud "operator"

Enumerator

OPER_ADD
OPER_SUB
OPER_MULT
OPER_DIV
OPER_SEQUENCE
OPER_TRANSITION
OPER_STEP
OPER_INF
OPER_SUP
OPER_GE
OPER_LE
OPER_NE
OPER_EQ
OPER_NOT
OPER_AND
OPER_OR
OPER_NEG
OPER_ASSIGN
OPER_RESERVE_SPACE
OPER_MAIN

Definition at line 24 of file ast.h.

4.2.5 Function Documentation

4.2.5.1 `nodeType* createIdentifierNode ( char * id, int flag )`

Definition at line 86 of file ast.c.

4.2.5.2 `nodeType* createNumericNode ( float v )`

Definition at line 30 of file ast.c.

4.2.5.3 `nodeType* createOperatorNode ( int oper, int nOperands, ... )`

Definition at line 57 of file ast.c.

4.2.5.4 `int Extract_Num ( char * str )`

Definition at line 598 of file ast.c.

4.2.5.5 `etapeNode* findEtapeByName ( etapeNode * head, char * s )`

Definition at line 374 of file ast.c.

4.2.5.6 `void generateAsmExpression ( nodeType * n, FILE * fout )`

Definition at line 113 of file ast.c.

4.2.5.7 `void generateCode ( etapeNode * en, transitionNode * tn )`

Definition at line 629 of file ast.c.

4.2.5.8 `void grafcet_interpreter ( )`

4.2.5.9 `void makeDual ( etapeNode * en, transitionNode * tn )`

Definition at line 448 of file ast.c.

4.2.5.10 `void printEtapes ( etapeNode * head )`

Definition at line 402 of file ast.c.

4.2.5.11 void printRefEtapes (etapeRefNode * head)

Definition at line 316 of file ast.c.

4.2.5.12 void printRefTransitions (transitionRefNode * head)

Definition at line 328 of file ast.c.

4.2.5.13 void printTransitions (transitionNode * head)

Definition at line 428 of file ast.c.

4.2.5.14 void pushAction (actionNode ** head, actionNode * action)

Definition at line 265 of file ast.c.

4.2.5.15 void pushEtape (etapeNode ** head, etapeNode * etape)

Definition at line 282 of file ast.c.

4.2.5.16 void pushRefEtape (etapeRefNode ** head, char * n)

Definition at line 219 of file ast.c.

4.2.5.17 void pushRefTransition (transitionRefNode ** head, char * n)

Definition at line 242 of file ast.c.

4.2.5.18 void pushTransition (transitionNode ** head, transitionNode * trans)

Definition at line 299 of file ast.c.

Index

- [_actionNode, 5](#)
 - [expr, 6](#)
 - [name, 6](#)
 - [next, 6](#)
 - [_etapeNode, 6](#)
 - [actions, 8](#)
 - [active, 8](#)
 - [id, 8](#)
 - [name, 8](#)
 - [nbTransitionsAmont, 8](#)
 - [nbTransitionsAval, 8](#)
 - [next, 8](#)
 - [transitionsAmont, 8](#)
 - [transitionsAval, 8](#)
 - [_etapeRefNode, 9](#)
 - [etape, 10](#)
 - [name, 10](#)
 - [next, 10](#)
 - [_transitionNode, 10](#)
 - [etapesAmont, 11](#)
 - [etapesAval, 11](#)
 - [id, 11](#)
 - [name, 11](#)
 - [nbEtapesAmont, 11](#)
 - [nbEtapesAval, 11](#)
 - [next, 11](#)
 - [receptivite, 11](#)
 - [_transitionRefNode, 12](#)
 - [name, 13](#)
 - [next, 13](#)
 - [transition, 13](#)
- [actionNode](#)
 - [ast.h, 25](#)
- [actions](#)
 - [_etapeNode, 8](#)
- [active](#)
 - [_etapeNode, 8](#)
- [ast.c, 19](#)
 - [compteurEtapes, 22](#)
 - [compteurLabel, 22](#)
 - [createIdentifierNode, 20](#)
 - [createNumericNode, 20](#)
 - [createOperatorNode, 20](#)
 - [Extract_Num, 20](#)
 - [findEtapeByName, 21](#)
 - [findTransitionByName, 21](#)
 - [generateAsmExpression, 21](#)
 - [generateCode, 21](#)
 - [makeDual, 21](#)
 - [nbEtapes, 21](#)
 - [nbTransitions, 21](#)
 - [num_variable, 22](#)
 - [printEtapes, 21](#)
 - [printRefEtapes, 21](#)
 - [printRefTransitions, 21](#)
 - [printTransitions, 22](#)
 - [pushAction, 22](#)
 - [pushEtape, 22](#)
 - [pushRefEtape, 22](#)
 - [pushRefTransition, 22](#)
 - [pushTransition, 22](#)
- [ast.h, 23](#)
 - [actionNode, 25](#)
 - [createIdentifierNode, 27](#)
 - [createNumericNode, 27](#)
 - [createOperatorNode, 27](#)
 - [etapeNode, 25](#)
 - [etapeRefNode, 25](#)
 - [Extract_Num, 27](#)
 - [findEtapeByName, 27](#)
 - [generateAsmExpression, 27](#)
 - [generateCode, 27](#)
 - [grafcet_interpreter, 27](#)
 - [KEY_ANA, 25](#)
 - [KEY_BUTTON, 25](#)
 - [KEY_IDENT, 25](#)
 - [KEY_PWM, 25](#)
 - [KEY_TOR, 25](#)
 - [makeDual, 27](#)
 - [nodeEnum, 26](#)
 - [nodeOper, 26](#)
 - [nodeType, 25](#)
 - [OPER_ADD, 26](#)
 - [OPER_AND, 26](#)
 - [OPER_ASSIGN, 26](#)
 - [OPER_DIV, 26](#)
 - [OPER_EQ, 26](#)
 - [OPER_GE, 26](#)
 - [OPER_INF, 26](#)
 - [OPER_LE, 26](#)
 - [OPER_MAIN, 26](#)
 - [OPER_MULT, 26](#)
 - [OPER_NEG, 26](#)
 - [OPER_NOT, 26](#)
 - [OPER_NE, 26](#)
 - [OPER_OR, 26](#)
 - [OPER_RESERVE_SPACE, 26](#)
 - [OPER_SEQUENCE, 26](#)

- OPER_STEP, 26
- OPER_SUB, 26
- OPER_SUP, 26
- OPER_TRANSITION, 26
- printEtapes, 27
- printRefEtapes, 27
- printRefTransitions, 28
- printTransitions, 28
- pushAction, 28
- pushEtape, 28
- pushRefEtape, 28
- pushRefTransition, 28
- pushTransition, 28
- transitionNode, 25
- transitionRefNode, 26
- typeIdentifier, 26
- typeNumeric, 26
- typeOperator, 26
- compteurEtapes
 - ast.c, 22
- compteurLabel
 - ast.c, 22
- createIdentifierNode
 - ast.c, 20
 - ast.h, 27
- createNumericNode
 - ast.c, 20
 - ast.h, 27
- createOperatorNode
 - ast.c, 20
 - ast.h, 27
- digraphNode
 - nodeTypeTag, 15
- etape
 - _etapeRefNode, 10
- etapeNode
 - ast.h, 25
- etapeRefNode
 - ast.h, 25
- etapesAmont
 - _transitionNode, 11
- etapesAval
 - _transitionNode, 11
- expr
 - _actionNode, 6
- Extract_Num
 - ast.c, 20
 - ast.h, 27
- findEtapeByName
 - ast.c, 21
 - ast.h, 27
- findTransitionByName
 - ast.c, 21
- flag
 - identifierNodeType, 13
- generateAsmExpression
 - ast.c, 21
 - ast.h, 27
- generateCode
 - ast.c, 21
 - ast.h, 27
- grafcet_interpreter
 - ast.h, 27
- id
 - _etapeNode, 8
 - _transitionNode, 11
- ident
 - identifierNodeType, 13
- identifierNodeType, 13
 - flag, 13
 - ident, 13
 - key, 13
- KEY_ANA
 - ast.h, 25
- KEY_BUTTON
 - ast.h, 25
- KEY_IDENT
 - ast.h, 25
- KEY_PWM
 - ast.h, 25
- KEY_TOR
 - ast.h, 25
- key
 - identifierNodeType, 13
- makeDual
 - ast.c, 21
 - ast.h, 27
- nOperands
 - operatorNodeType, 16
- name
 - _actionNode, 6
 - _etapeNode, 8
 - _etapeRefNode, 10
 - _transitionNode, 11
 - _transitionRefNode, 13
- nbEtapes
 - ast.c, 21
- nbEtapesAmont
 - _transitionNode, 11
- nbEtapesAval
 - _transitionNode, 11
- nbTransitions
 - ast.c, 21
- nbTransitionsAmont
 - _etapeNode, 8
- nbTransitionsAval
 - _etapeNode, 8
- next
 - _actionNode, 6
 - _etapeNode, 8

- [_etapeRefNode](#), 10
 - [_transitionNode](#), 11
 - [_transitionRefNode](#), 13
- [nodeEnum](#)
 - [ast.h](#), 26
- [nodeOper](#)
 - [ast.h](#), 26
- [nodeType](#)
 - [ast.h](#), 25
- [nodeTypeTag](#), 14
 - [digraphNode](#), 15
 - [t_identifier](#), 15
 - [t_numeric](#), 15
 - [t_oper](#), 15
 - [type](#), 15
- [num_variable](#)
 - [ast.c](#), 22
- [numericNodeType](#), 15
 - [valeur](#), 16
- [OPER_ADD](#)
 - [ast.h](#), 26
- [OPER_AND](#)
 - [ast.h](#), 26
- [OPER_ASSIGN](#)
 - [ast.h](#), 26
- [OPER_DIV](#)
 - [ast.h](#), 26
- [OPER_EQ](#)
 - [ast.h](#), 26
- [OPER_GE](#)
 - [ast.h](#), 26
- [OPER_INF](#)
 - [ast.h](#), 26
- [OPER_LE](#)
 - [ast.h](#), 26
- [OPER_MAIN](#)
 - [ast.h](#), 26
- [OPER_MULT](#)
 - [ast.h](#), 26
- [OPER_NEG](#)
 - [ast.h](#), 26
- [OPER_NOT](#)
 - [ast.h](#), 26
- [OPER_NE](#)
 - [ast.h](#), 26
- [OPER_OR](#)
 - [ast.h](#), 26
- [OPER_RESERVE_SPACE](#)
 - [ast.h](#), 26
- [OPER_SEQUENCE](#)
 - [ast.h](#), 26
- [OPER_STEP](#)
 - [ast.h](#), 26
- [OPER_SUB](#)
 - [ast.h](#), 26
- [OPER_SUP](#)
 - [ast.h](#), 26
- [OPER_TRANSITION](#)
 - [ast.h](#), 26
- [op](#)
 - [operatorNodeType](#), 16
- [oper](#)
 - [operatorNodeType](#), 17
- [operatorNodeType](#), 16
 - [nOperands](#), 16
 - [op](#), 16
 - [oper](#), 17
- [printEtapes](#)
 - [ast.c](#), 21
 - [ast.h](#), 27
- [printRefEtapes](#)
 - [ast.c](#), 21
 - [ast.h](#), 27
- [printRefTransitions](#)
 - [ast.c](#), 21
 - [ast.h](#), 28
- [printTransitions](#)
 - [ast.c](#), 22
 - [ast.h](#), 28
- [pushAction](#)
 - [ast.c](#), 22
 - [ast.h](#), 28
- [pushEtape](#)
 - [ast.c](#), 22
 - [ast.h](#), 28
- [pushRefEtape](#)
 - [ast.c](#), 22
 - [ast.h](#), 28
- [pushRefTransition](#)
 - [ast.c](#), 22
 - [ast.h](#), 28
- [pushTransition](#)
 - [ast.c](#), 22
 - [ast.h](#), 28
- [receptivite](#)
 - [_transitionNode](#), 11
- [t_identifier](#)
 - [nodeTypeTag](#), 15
- [t_numeric](#)
 - [nodeTypeTag](#), 15
- [t_oper](#)
 - [nodeTypeTag](#), 15
- [transition](#)
 - [_transitionRefNode](#), 13
- [transitionNode](#)
 - [ast.h](#), 25
- [transitionRefNode](#)
 - [ast.h](#), 26
- [transitionsAmont](#)
 - [_etapeNode](#), 8
- [transitionsAval](#)
 - [_etapeNode](#), 8
- [type](#)
 - [nodeTypeTag](#), 15

typeIdentifier

ast.h, [26](#)

typeNumeric

ast.h, [26](#)

typeOperator

ast.h, [26](#)

valeur

numericNodeType, [16](#)